

编译原理 第一次作业

姓名：朱首赫

学号：2023K8009906029

1 练习 1.1.1

编译器相对于解释器的优点是什么？解释器相对于编译的优点是什么？

编译器相对于解释器的优点：

1. 运行速度极快：编译器将源代码一次性转换成机器语言，执行时不需要再次翻译，速度远快于解释器。
2. 避免重复翻译：程序编译一次后可以重复运行，不需要重复进行翻译操作。
3. 优化能力强：编译器在翻译时可以掌握全局信息，从而更好地进行代码优化，而解释器只能逐行执行。

解释器相对于编译器的优点：

1. 使用便捷：省去了编译的步骤，可以直接运行。
2. 便于调试：可以精确定位报错位置，并且比较容易实现交互式编程，便于调试或进行实验。
2. 可移植性：只需要目标设备有对应的解释器，同一份代码基本都可以跨平台运行，无需重新编译。

2 练习 1.1.2

在一个语言处理系统中，编译器产生汇编语言而不是机器语言的好处是什么？

为了利用已经成熟的汇编器去完成底层操作细节，从而简化编译器的设计与实现。此外，输出汇编代码远比机器语言更有可读性，便于进行编译器调试。

3 练习 1.1.3

对下图1中的块结构的 C 代码，指出每次赋给 w、x、y 和 z 的值

(1) <pre>int w, x, y, z; int i = 5, j = 13; { int j = 17; i = j - 10; w = i + j; } x = i * j; { int i = j; y = i * j; } z = i + j;</pre>	1	(2) <pre>int w, x, y, z; int i = 9, j = 7; { int i = j + 1; w = i * 2; } x = i - j; { int j = 17; i = 11; y = i + j; } z = i + j;</pre>
---	----------	--

图 1

(1) w=24, x=91, y=169, z=20

(2) w=16, x=2, y=28, z=18

4 练习 1.1.4

下面的 C 代码的打印结果是什么？

```
1 #include <stdio.h>
2 #define a x
3     int x = 4;
4     void b( ) { x = a * 3; printf( "%d\n" ,x); }
5     void c( ) { int x = 1111; printf( "%d\n" , a+1); }
6     void main( ) { b( ); c( ); }
```

打印结果为：

12

1112

5 练习 1.1.5

有人把程序设计语言分为编译型和解释型两类，例如 C 是编译型，Python 是解释型。这个分类是否合理？能否构建 C 语言的解释器，或者 Python 的静态编译器？谈谈你的看法。

这个分类并不合理。“编译”或“解释”是程序执行流实现方式，而不是语言本身的属性。语言本身是串的集合，具体到程序设计语言则是所有符合语法规则的程序集合。而编译器和解释器使代码可运行的实现工具，任何程序设计语言理论上既可以被编译也可以被解释。

可以构建 C 语言的解释器，比如现有的 Cling、Ch 等解释器，可用于快速开发和实验性工作。同样可以构建 Python 的静态编译器，比如 Cython 可以将 Python 代码编译为 C 扩展，大大提升运行速度。